WebX-2025-May-PYQ Answers

Q1. [5 Marks each]

a. What is Python Flask?

b. Explain AngularJS Scope.

c. What are the features of MongoDB?

d. State the features of TypeScript.

e. Explain different types of Web Analytics.

Q2. [10 Marks each]

a. Explain Arithmetic operators with suitable example.

b. How do you set, access and delete cookies in Python Flask?

Q3. [10 Marks each]

a. Explain Flask templates with an example.

b. Explain different characteristics of RIA in details.

Q4. [10 Marks each]

a. Explain REST API in detail.

b. Explain Drupal's architecture with its advantages.

Q5. [10 Marks each]

a. What is an AngularJS dependency injection? State 6 examples of AngularJS built-in helper function.

b. Explain MongoDB Data Types along with syntax.

Q6. [10 Marks each]

a. List the features of AngularJS. Also state its advantages and disadvantages.

b. Explain how internal and External Modules are used in TypeScript with suitable example.

Q1. [5 Marks each] - Answers

a. What is Python Flask?

- Flask is a lightweight **web framework** written in Python.
- It is based on the **WSGI (Web Server Gateway Interface)** standard.
- Provides tools to build **web applications and APIs** quickly.
- Known as a **"micro-framework"** because it does not include built-in components like ORM or form validation.
- Developers can extend Flask using **extensions** (e.g., for database, authentication).
- It supports **routing**, **templating (Jinja2)**, and **request handling**.

b. Explain AngularJS Scope.

1. Definition

Scope in AngularJS is an object that refers to the application model. It acts as a bridge between the controller and the view (HTML).

2. Key Points

- Scope is a JavaScript object used to hold data and functions.
- Controllers set values/functions on scope, which can be accessed in the view.
- Scope supports two-way data binding (changes in model reflect in view and vice versa).
- It can be hierarchical — child scopes inherit from parent scopes.
- `$scope` is injected into controllers by AngularJS.
- Digest cycle updates the view whenever scope data changes.

3. Example / Code

```
// Controller definition
app.controller("myCtrl", function($scope) {
    $scope.message = "Hello AngularJS"; // data stored in sco
pe
});

// HTML view
<div ng-controller="myCtrl">
    {{ message }} <!-- binds scope data to view -->
</div>
```

c. What are the features of MongoDB?

1. Definition:

MongoDB is a NoSQL, document-oriented database that stores data in flexible JSON-like documents. It is designed for high performance, scalability, and easy data handling.

2. Key Features of MongoDB:

- **Document-Oriented Storage:** Stores data in BSON documents instead of tables and rows.
- **Schema-less Database:** No fixed structure, allowing flexible and dynamic data models.
- **High Performance:** Provides fast read and write operations.
- **Scalability:** Supports horizontal scaling through sharding.
- **Indexing Support:** Allows indexing to improve query performance.
- **Aggregation Framework:** Supports powerful data processing and analysis operations.

d. State the features of TypeScript.

1. Definition

TypeScript is a superset of JavaScript developed by Microsoft that adds optional static typing and advanced features to make large-scale application development easier.

2. Key Points

- **Static Typing**: Allows defining types for variables, functions, and objects.
- **Object-Oriented Features**: Supports classes, interfaces, and inheritance.
- **Compile-Time Error Checking**: Detects errors before running the code.
- **Compatibility**: Fully compatible with JavaScript; TypeScript code compiles to plain JavaScript.
- **Tooling Support**: Provides IntelliSense, autocompletion, and better IDE support.
- **Modularity**: Supports modules for structured and maintainable code.
- **Scalability**: Suitable for building large, complex applications.

e. Explain different types of Web Analytics.

1. Definition:

Web Analytics is the process of collecting, measuring, analyzing, and reporting website data to understand user behavior and improve website performance.

2. Types of Web Analytics:

- **On-Site Web Analytics:** Measures visitor activity on a website such as page views, bounce rate, and session duration.
- **Off-Site Web Analytics:** Analyzes a website’s presence outside the site, such as social media reach and competitor analysis.
- **Descriptive Analytics:** Shows what has happened on the website using reports and statistics.

- **Diagnostic Analytics:** Explains why something happened by analyzing user behavior patterns.
- **Predictive Analytics:** Uses past data to predict future trends and user actions.
- **Prescriptive Analytics:** Suggests actions to improve website performance based on analysis.
- **Real-Time Analytics:** Tracks and reports website activity instantly as users interact.

Q2. [10 Marks each] - Answers

a. Explain Arithmetic operators with suitable example.

Definition

Arithmetic operators are symbols used to perform mathematical operations on operands (variables or values). They are commonly used in programming languages for calculations.

Types of Arithmetic Operators

1. **Addition (+)**
   - Used to add two operands.
   - Syntax: `a + b`
2. **Subtraction (-)**
   - Used to subtract one operand from another.
   - Syntax: `a - b`
3. **Multiplication (*)**
   - Used to multiply two operands.
   - Syntax: `a * b`
4. **Division (/)**
   - Used to divide one operand by another.

- Syntax: `a / b`

5. **Modulus (%)**
   - Returns the remainder after division.
   - Syntax: `a % b`
6. **Increment (++)**
   - Increases the value of a variable by 1.
   - Example: `a++`
7. **Decrement (--)**
   - Decreases the value of a variable by 1.
   - Example: `a--`

Example / Code

```
let a = 10;
let b = 5;

console.log(a + b); // Addition = 15
console.log(a - b); // Subtraction = 5
console.log(a * b); // Multiplication = 50
console.log(a / b); // Division = 2
console.log(a % b); // Modulus = 0

a++; // Increment
b--; // Decrement

console.log(a); // 11
console.log(b); // 4
```

b. How do you set, access and delete cookies in Python Flask?

Definition

Cookies are small pieces of data stored in the user's browser by the server. In Python Flask, cookies are used to store user-specific information such as login details, preferences, and session data.

1. Setting Cookies in Flask

- Cookies are set using the `set_cookie()` method.
- It is applied to the response object before sending it to the client.
- Syntax:

```
response.set_cookie('key', 'value')
```

Steps to Set a Cookie:

1. Create a response object.
2. Use `set_cookie()` to store data.

2. Accessing Cookies in Flask

- Cookies stored in the browser can be accessed using the `request.cookies.get()` method.
- It retrieves the value associated with a cookie name.

Syntax:

```
request.cookies.get('key')
```

Steps to Access a Cookie:

1. Import `request` from Flask.
2. Use `request.cookies.get()` to read cookie data.

3. Deleting Cookies in Flask

- Cookies can be deleted using `delete_cookie()` method.
- It removes the cookie from the browser.

Syntax:

```
response.delete_cookie('key')
```

Steps to Delete a Cookie:

1. Create a response object.
2. Use `delete_cookie()` with the cookie name.

Example / Code

```
from flask import Flask, request, make_response

app = Flask(__name__)

# Setting a cookie
@app.route('/set')
def setcookie():
    res = make_response("Cookie Set")
    res.set_cookie('username', 'Sameer')
    return res

# Accessing a cookie
@app.route('/get')
def getcookie():
    user = request.cookies.get('username')
    return "Username is " + user

# Deleting a cookie
@app.route('/delete')
def deletecookie():
    res = make_response("Cookie Deleted")
    res.delete_cookie('username')
    return res
```

Q3. [10 Marks each] - Answers

a. Explain Flask templates with an example.

Definition

Flask templates are HTML files used to display dynamic content in Flask applications. Flask uses the Jinja template engine, which allows embedding Python-like expressions and control statements inside HTML.

What are Flask Templates?

- Templates separate application logic from presentation.
- They help generate dynamic web pages.
- Template files are usually stored inside a **templates** folder.
- Flask uses the `render_template()` function to load and display templates.

Features of Flask Templates

1. **Dynamic Content Rendering**
   - Variables can be passed from Flask to HTML.
   - Example: displaying username, age, etc.
2. **Control Statements**
   - Supports conditions using `if`.
   - Supports loops using `for`.
3. **Template Inheritance**
   - One base template can be reused by other pages.
   - Reduces code duplication.
4. **Separation of Concerns**
   - Python code stays in Flask file.

- HTML design stays in template files.

Working of Flask Templates

1. User requests a webpage.
2. Flask route processes the request.
3. Data is passed to template using `render_template()`.
4. Template generates dynamic HTML and sends it to browser.

Example / Code

Flask Application (app.py)

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html', name="The Office")
```

Template File (templates/index.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Flask Template</title>
</head>
<body>
    <h1>Welcome to {{ name }}</h1> <!-- Dynamic variable -->
</body>
</html>
```

b. Explain different characteristics of RIA in details.

Definition

A **Rich Internet Application (RIA)** is a browser-based application that delivers the speed, interactivity, and responsiveness of desktop software while being accessible online. Unlike static websites, RIAs provide real-time updates, animations, and smooth user interactions.

Characteristics of RIAs

1. **Enhanced User Experience**
    - Provides visually rich interfaces with animations, drag-and-drop, and responsive feedback.
    - Mimics desktop-like usability.
2. **Dynamic Content**
    - Updates data in real-time without reloading the entire page.
    - Uses AJAX and JavaScript for asynchronous communication.
3. **Offline Capabilities**
    - Can cache data locally, allowing limited functionality without internet connectivity.
4. **Better Performance**
    - Reduces server load by handling more tasks on the client side.
    - Improves speed and responsiveness.
5. **Cross-Platform Accessibility**
    - Runs inside standard web browsers, independent of operating system.
    - Accessible across devices (desktop, mobile, tablets).
6. **Integration with Backend Services**
    - Connects with APIs and databases for real-time data exchange.
    - Common in e-commerce, banking, healthcare, and SaaS platforms.

7. **Security Features**
    - Requires strong authentication, encryption, and authorization to protect user data.

Example

- **Google Docs/Sheets** → Real-time collaboration with instant updates.
- **Netflix** → Interactive menus, previews, and seamless streaming.
- **Salesforce CRM**

Q4. [10 Marks each] - Answers

a. Explain REST API in detail.

Definition

A **REST API (Representational State Transfer API)** is a web service interface that allows communication between client and server using standard HTTP methods. It follows REST architecture principles, enabling applications to perform **CRUD (Create, Read, Update, Delete)** operations on resources identified by unique URLs.

Key Characteristics of REST API

1. **Statelessness** – Each request contains all necessary information; the server does not store client state.
2. **Client-Server Separation** – Client handles UI, server manages data and logic.
3. **Uniform Interface** – Resources accessed via standard HTTP methods (GET, POST, PUT, PATCH, DELETE).
4. **Resource Identification** – Each resource is identified by a unique URI.
5. **Representation** – Data exchanged in formats like JSON or XML.
6. **Cacheable Responses** – Responses specify whether they can be cached to improve performance.

HTTP Methods in REST API

- **GET** → Retrieve resource (e.g., `GET /users/123`)
- **POST** → Create new resource (e.g., `POST /users`)
- **PUT** → Update/replace entire resource (e.g., `PUT /users/123`)
- **PATCH** → Partially update resource (e.g., `PATCH /users/123`)
- **DELETE** → Remove resource (e.g., `DELETE /users/123`)

Example (Python Flask REST API)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

users = {"1": {"name": "Michael Scott", "email": "prisonmike@
dm.com"}}

@app.route('/users/<id>', methods=['GET'])
def get_user(id):
    return jsonify(users.get(id))  # GET → Read

@app.route('/users', methods=['POST'])
def create_user():
    data = request.json
    users["2"] = data
    return jsonify({"message": "User created"}), 201  # POST
→ Create

@app.route('/users/<id>', methods=['PUT'])
def update_user(id):
    users[id] = request.json
    return jsonify({"message": "User updated"})  # PUT → Upda
te
```

```python
@app.route('/users/<id>', methods=['DELETE'])
def delete_user(id):
    users.pop(id, None)
    return jsonify({"message": "User deleted"})  # DELETE → D
elete
```

b. Explain Drupal's architecture with its advantages.

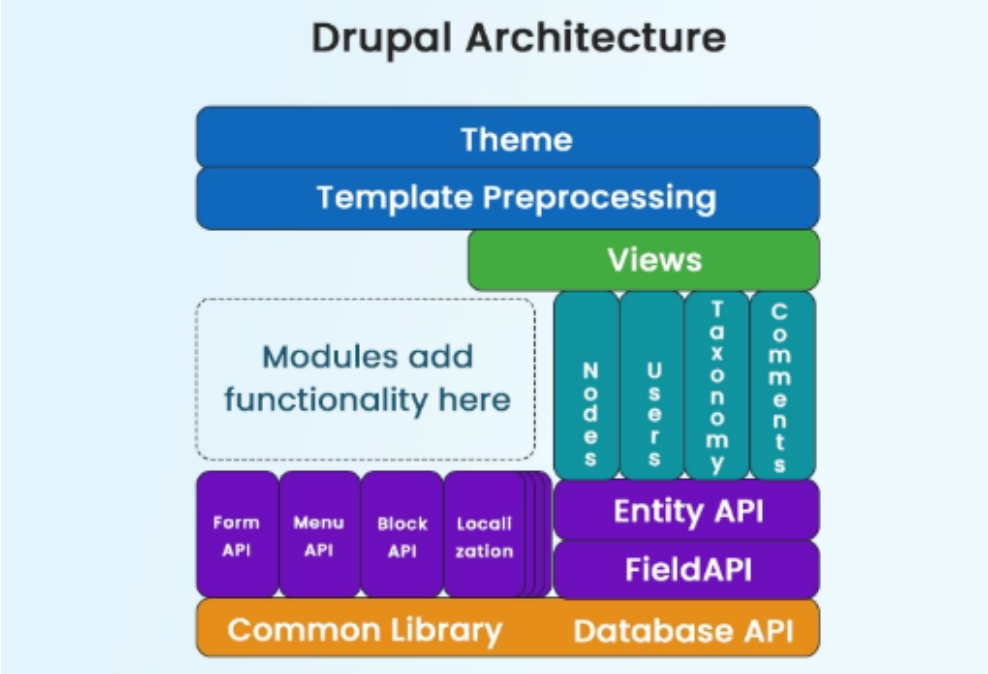
Definition

Drupal is an open-source Content Management System (CMS) used to build dynamic websites and applications. Its architecture is designed to handle complex workflows, large-scale content, and integration with external systems.

Drupal Architecture

1. **Users Layer**
   - End-users interact with Drupal through browsers or search engines.
   - Requests are sent via HTTP to the web server.
2. **Administrator Layer**
   - Admins manage permissions, block unauthorized access, and control site configuration.
   - Full privileges for content and workflow management.
3. **Drupal Core Layer**
   - Built on PHP, provides modules, themes, and APIs.
   - Handles content organization, publishing, and customization.
   - Uses **hooks, services, and plugins** for extensibility.
4. **PHP Layer**
   - Executes application logic and interacts with the web server.
   - Memory requirements vary by version (Drupal 6 → 16MB, Drupal 7 → 32MB, Drupal 8 → 64MB).

5. **Web Server Layer**
    - Processes requests via HTTP and serves files to users.
    - Common servers: Apache, Nginx, IIS.
6. **Database Layer**
    - Stores user data, content, and configuration.
    - Supports MySQL, PostgreSQL, SQLite, etc.
    - Enables CRUD operations and caching.



Advantages of Drupal

- **Modular Architecture** → Highly customizable with thousands of modules and plugins.
- **Scalability** → Handles high traffic and complex data structures efficiently.
- **Security** → Role-based access control, frequent updates, and strong community support.

- **Multilingual Support** → Built-in translation workflows for global websites.
- **Low Cost of Ownership** → Open-source, no vendor lock-in, supported by a large ecosystem.

Q5. [10 Marks each] - Answers

a. What is an AngularJS dependency injection? State 6 examples of AngularJS built-in helper function.

Part A: Dependency Injection in AngularJS

Definition

Dependency Injection (DI) in AngularJS is a **design pattern** where components (controllers, services, directives) are automatically provided with their required dependencies by the AngularJS **injector subsystem**, instead of creating them manually.

Explanation

- Promotes **loose coupling** between components.
- Improves **modularity** and **testability**.
- Dependencies can be values, services, factories, or providers.
- AngularJS automatically resolves and injects them into components.

Example

```
var app = angular.module("mainApp", []);

app.service("MathService", function() {
  this.square = function(x) { return x * x; };
});

app.controller("CalcController", function($scope, MathServic
e) {
  $scope.num = 4;
  $scope.result = MathService.square($scope.num); // Dependen
```

```
cy injected
});
```

Part B: Six Examples of AngularJS Built-in Helper Functions

1. **angular.isArray(obj)** → Checks if the given object is an array.
2. **angular.isDate(obj)** → Checks if the object is a valid date.
3. **angular.isDefined(value)** → Returns true if the value is defined.
4. **angular.isElement(obj)** → Checks if the object is a DOM element.
5. **angular.isNumber(value)** → Returns true if the value is a number.
6. **angular.isString(value)** → Returns true if the value is a string.

b. Explain MongoDB Data Types along with syntax.

Definition

MongoDB is a **NoSQL document-oriented database** that stores data in **BSON (Binary JSON)** format. BSON supports a wide range of data types beyond standard JSON, making MongoDB flexible for handling structured and semi-structured data.

Common MongoDB Data Types with Syntax

1. **String**
   - Stores text data.
   - Syntax: `"field": "value"`
2. **Integer**
   - Stores numeric values (32-bit or 64-bit depending on system).
   - Syntax: `"field": 123`
3. **Boolean**
   - Stores `true` or `false`.

- Syntax: `"field": true`

4. **Double**
   - Stores floating-point numbers.
   - Syntax: `"field": 12.34`
5. **Array**
   - Stores multiple values in a single field.
   - Syntax: `"field": [1, 2, 3]`
6. **Object / Embedded Document**
   - Stores nested documents.
   - Syntax: `"field": { "subfield": "value" }`
7. **Null**
   - Represents an empty or missing value.
   - Syntax: `"field": null`
8. **ObjectId**
   - Unique identifier automatically generated for each document.
   - Syntax: `"_id": ObjectId("507f1f77bcf86cd799439011")`
9. **Date**
   - Stores date and time values.
   - Syntax: `"field": new Date("2026-04-26")`

Example (MongoDB Document Syntax):

```
db.students.insertOne({
    _id: ObjectId("507f1f77bcf86cd799439011"), // Unique ID
    name: "Dwight Schrute",                    // String
    age: 55,                                   // Integer
    enrolled: true,                            // Boolean
    sales: [85, 90, 92],                       // Array
```

```
    address: { city: "Pennsylvania", pin: 19061}, // Embedded
Document
    promotionDate: new Date("2009-10-01"),    // Date
    profilePic: null                          // Null
});
```

Q6. [10 Marks each] - Answers

a. List the features of AngularJS. Also state its advantages and disadvantages.

Features of AngularJS

1. **Two-Way Data Binding** → Synchronizes data between model and view automatically.
2. **Directives** → Extend HTML with custom attributes (`ng-model`, `ng-repeat`, etc.).
3. **MVC Architecture** → Separates application logic, data, and presentation.
4. **Dependency Injection** → Provides required services/components automatically.
5. **Routing** → Enables navigation between views without reloading the page.
6. **Testing Support** → Built-in support for unit testing and end-to-end testing.

Advantages of AngularJS

- **Simplifies Development** → Less code due to data binding and directives.
- **Reusable Components** → Modular design with services and directives.
- **Dynamic and Responsive UI** → Real-time updates without page reloads.
- **Strong Community Support** → Backed by Google and large developer community.
- **Cross-Platform** → Works across browsers and devices.

Disadvantages of AngularJS

- **Performance Issues** → Slower with large, complex applications due to heavy DOM manipulation.
- **Learning Curve** → Concepts like directives, dependency injection, and scopes can be difficult for beginners.
- **Verbose Code** → Sometimes requires more configuration compared to lightweight frameworks.

b. Explain how internal and External Modules are used in TypeScript with suitable example.

Definition

In TypeScript, modules are used to organize code into separate reusable units. They help manage large applications by dividing code into smaller parts. TypeScript supports **Internal Modules** and **External Modules.**

1. Internal Modules (Namespace)

- Internal modules are called **Namespaces** in TypeScript.
- They are used to group related classes, functions, and interfaces within a single program.
- Declared using the `namespace` keyword.
- Mainly used for organizing code internally.

Syntax

```
namespace ModuleName {
   export class ClassName {
      // code
   }
}
```

Example

```
namespace EmployeeModule {

   export class Employee {
      name:string = "Jim Halpert";

      display() {
         console.log(this.name);
      }
   }

}

let obj = new EmployeeModule.Employee();
obj.display();
```

Uses of Internal Modules

- Organizes code logically
- Avoids naming conflicts
- Improves code management

2. External Modules

- External modules are used when code is stored in separate files.
- They use `export` and `import` keywords.
- Commonly used in large applications.

File 1: student.ts

```
export class Employee {
   name:string = "Jim Halpert";

   display() {
      console.log(this.name);
```

```
    }
}
```

File 2: app.ts

```
import {Employee} from './employee';

let obj = new Employee();
obj.display();
```

Uses of External Modules

- Reusable code across files
- Better modular programming
- Easy maintenance
- Suitable for large projects

WebX-2025-December-PYQ Answers

Q1. [5 Marks each]

a. Compare and contrast Web 1.0, Web 2.0 and Web 3.0?

b. Explain AngularJS Controller.

c. What are the features of MongoDB?

d. Differentiate between JavaScript and TypeScript.

e. What is Content Management System (CMS)?

Q2. [10 Marks each]

a. Define Semantic Web. Explain the components of Semantic Web Stack with diagram.

b. Explain how internal and External Modules are used in TypeScript with suitable example.

Q3. [10 Marks each]

a. Explain AngularJS filters with example.

b. Explain Flask templates with an example.

Q4. [10 Marks each]

a. Explain Joomla's architecture with all 7 components. Drupal allows you to create three kinds of contents: state them.

b. Explain REST API in detail.

Q5. [10 Marks each]

a. Explain MongoDB CRUD Operations with an example.

b. What is an AngularJS dependency injection? State 6 examples of AngularJS built-in helper function.

Q6. [10 Marks each]

a. How to set, access and delete cookies in Python Flask?

b. With the help of diagram explain process involved in AJAX Asynchronous Model.

Q1. [5 Marks each] - Answers

a. Compare and contrast Web 1.0, Web 2.0 and Web 3.0?

Feature	Web 1.0	Web 2.0	Web 3.0
Meaning	Static Web	Social/Interactive Web	Semantic/Intelligent Web
Nature	Read-only	Read and Write	Read, Write and Execute
User Interaction	Very limited	High user interaction	Intelligent interaction
Content	Static content	Dynamic and user-generated content	Machine-understandable linked data
Role of Users	Consumers only	Consumers and contributors	Users plus intelligent agents
Communication	One-way communication	Two-way communication	Multi-way intelligent communication
Data Handling	Isolated data	Shared data	Linked and semantic data
Technologies Used	HTML, HTTP, URLs	AJAX, XML, JavaScript, APIs	RDF, OWL, AI, Semantic Web
Intelligence	No intelligence	Limited intelligence	High intelligence
Personalization	Very low	Moderate	High
Examples	Static websites	Blogs, Wikis, Social Media	Smart assistants, Semantic search

b. Explain AngularJS Controller.

Definition

In AngularJS, a **Controller** is a JavaScript function that manages the **data and logic** of an application. It acts as the **business logic layer**, connecting the **view (HTML)** with the **model (data)** using the `$scope` object.

Working of Controllers

- Controllers are defined using `app.controller()`.
- `$scope` is injected into the controller, allowing data and functions to be attached.
- The view (HTML) accesses `$scope` variables/functions using AngularJS expressions (`{{ }}`).
- Supports **two-way data binding**: changes in the model reflect in the view and vice versa.

Example – AngularJS Controller

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>AngularJS Controller Example</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularj
s/1.8.2/angular.min.js"></script>
  <script>
    var app = angular.module("myApp", []);
    app.controller("studentCtrl", function($scope) {
        $scope.name = "Ali";        // model data
        $scope.course = "Web X.0";    // model data
        $scope.showDetails = function() {
            return $scope.name + " is enrolled in " + $scope.
course;
        };
    });
  </script>
```

```
</head>
<body ng-controller="studentCtrl">
  <p>Student Name: {{name}}</p>
  <p>Course: {{course}}</p>
  <p>Details: {{showDetails()}}</p>
</body>
</html>
```

Explanation

- `studentCtrl` is a controller managing data (`name`, `course`).
- `$scope` binds these values to the view.
- `{{name}}` and `{{course}}` display data dynamically.
- `showDetails()` demonstrates how functions can be defined in the controller and used in the view.

c. Differentiate between JavaScript and TypeScript.

Feature	JavaScript	TypeScript
Type	Scripting language	Superset of JavaScript
Typing	Dynamic typing	Static typing
Error Checking	Errors found at runtime	Errors found at compile time
Compilation	No compilation needed	Compiled into JavaScript
OOP Support	Limited support	Strong support (classes, interfaces)
Code Maintenance	Harder for large projects	Easier for large projects
Performance in Development	Faster to write	Better debugging and development support

e. What is Content Management System (CMS

1. Definition:

Content Management System (CMS) is software used to create, manage, edit, and publish digital content on a website without needing advanced programming knowledge.

2. Key Points:

- It allows users to create and update website content easily.
- Provides a user-friendly interface for managing pages, posts, and media.
- Supports content editing, publishing, and deletion.
- Offers templates and themes for website design.
- Includes user management with roles and permissions.
- Supports plugins/extensions to add extra features.
- Makes website maintenance faster and easier.

3. Example:

Examples of CMS include WordPress, Joomla, and Drupal.

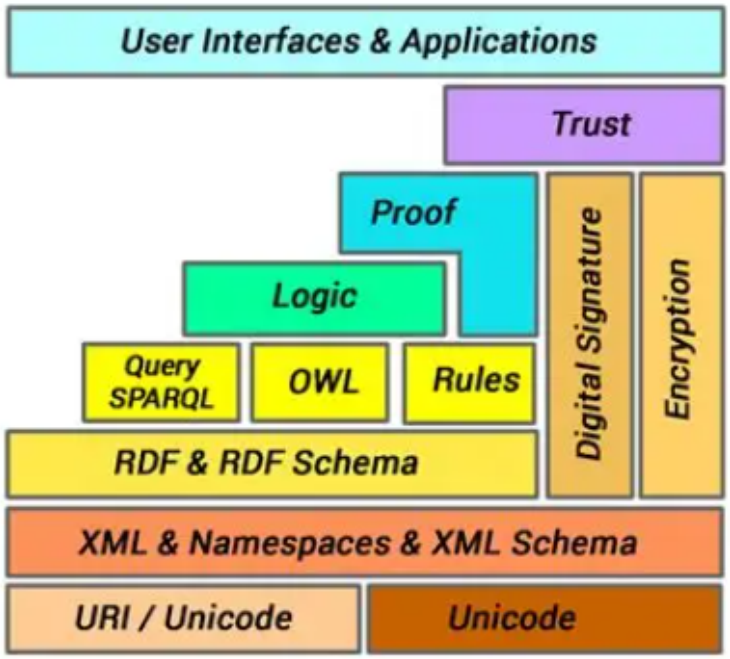
Q2. [10 Marks each] - Answers

a. Define Semantic Web. Explain the components of Semantic Web Stack with diagram.

Definition

The **Semantic Web**, proposed by Tim Berners-Lee, is a vision of the web where data is structured and linked in a way that allows computers to interpret meaning. It enables intelligent search, integration, and reasoning over web content.

The **Semantic Web Stack (Layer Cake)** is a layered architecture showing how different standards and technologies build upon each other to achieve this vision.



Components of Semantic Web Stack

1. **Foundation Layer**
   - **Unicode & URI/IRI** → Provide universal identifiers and multilingual text support.
   - **XML & XML Namespaces** → Enable structured representation of semi-structured data.
2. **Data Representation Layer**
   - **RDF (Resource Description Framework)** → Represents data as triples (subject–predicate–object).
   - **RDFS (RDF Schema)** → Provides vocabulary and relationships for RDF data.
3. **Ontology Layer**
   - **OWL (Web Ontology Language)** → Defines complex relationships, classes, and ontologies for richer semantics.
4. **Query Layer**

- **SPARQL** → Query language for RDF data, enabling retrieval and manipulation of semantic information.

5. **Logic & Rules Layer**
   - Allows reasoning over data, inferring new facts from existing ones.
6. **Proof Layer**
   - Ensures validity of reasoning processes and supports verifiable conclusions.
7. **Trust Layer**
   - Mechanisms for verifying sources, ensuring authenticity and reliability of data.

Example

- RDF Triple: `<Delhi> <capitalOf> <India>`
- With OWL: Define rule → *If X is a capital of Y, then X is a city.*
- SPARQL Query: *Find all capitals of countries in Asia.*

Q3. [10 Marks each] - Answers

a. Explain AngularJS filters with example.

Definition

Filters in AngularJS are used to **format and transform data** before displaying it in the view. They can be applied in templates, directives, or controllers to modify how data appears without changing the underlying model.

Common AngularJS Filters

1. **currency** → Formats a number as currency.
2. **date** → Formats a date to a specified format.
3. **filter** → Filters an array based on a condition.

4. **json** → Displays an object in JSON format.
5. **limitTo** → Limits an array/string to a specified number of elements.
6. **lowercase / uppercase** → Converts text to lower or upper case.
7. **orderBy** → Orders an array by a specified expression.

Usage

- Filters are applied using the pipe (`|`) symbol in AngularJS expressions.
- Syntax: `{{ expression | filterName:parameter }}`

Example

HTML Template

```
<div ng-app="myApp" ng-controller="myCtrl">
  <p>Currency: {{ price | currency }}</p>
  <p>Date: {{ today | date:'fullDate' }}</p>
  <p>Uppercase: {{ name | uppercase }}</p>
  <p>Limit: {{ items | limitTo:3 }}</p>
</div>
```

Controller

```
var app = angular.module("myApp", []);
app.controller("myCtrl", function($scope) {
    $scope.price = 2500;
    $scope.today = new Date();
    $scope.name = "Ryan Howard";
    $scope.items = ["AngularJS", "React", "Vue", "NodeJS"];
});
```

Output

- Currency: ₹2,500.00
- Date: Sunday, April 26, 2026

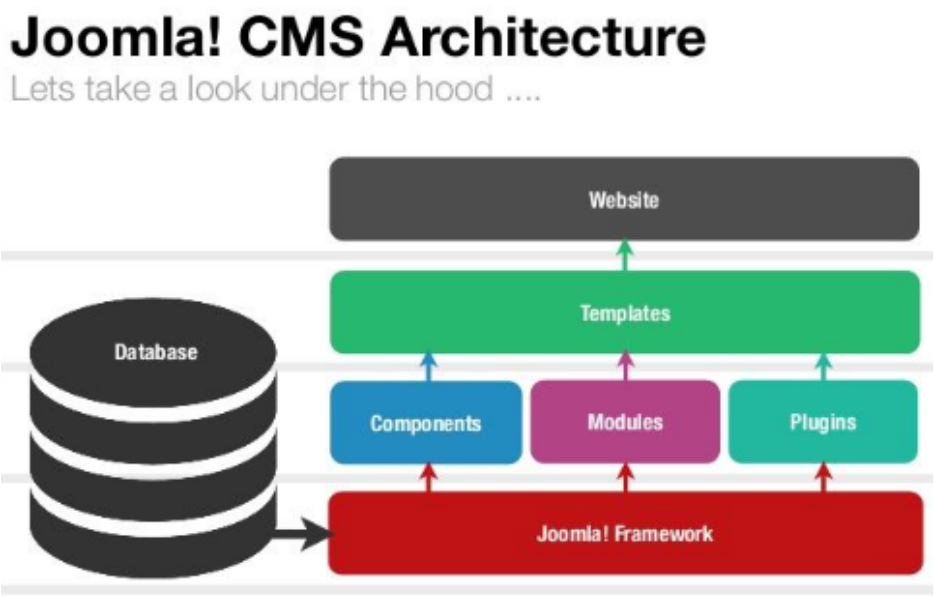
- Uppercase: RYAN HOWARD
- Limit: AngularJS, React, Vue

Q4. [10 Marks each] - Answers

a. Explain Joomla's architecture with all 7 components. Drupal allows you to create three kinds of contents: state them.

Definition

Joomla is an open-source Content Management System (CMS) used to build websites and online applications. Its architecture is modular and consists of **7 main components** that manage content, users, extensions, and system functionality.



7 Components of Joomla Architecture

1. **Content Management**
    - Handles articles, categories, and media.
    - Provides WYSIWYG editor for publishing.

2. **User Management**
    - Manages user accounts, roles, and permissions.
    - Supports access control levels (ACL).
3. **Menu Management**
    - Creates and organizes navigation menus.
    - Links menus to articles, components, or external URLs.
4. **Template Management**
    - Controls the design and layout of the site.
    - Allows switching between templates easily.
5. **Extension Management**
    - Supports plugins, modules, and components.
    - Extends Joomla's core functionality.
6. **Language Management**
    - Provides multilingual support.
    - Enables translation of site content and interface.
7. **System Management**
    - Handles configuration, caching, and updates.
    - Ensures site performance and security.

Drupal Content Types

Drupal allows you to create **three kinds of content**:

1. **Pages** → Static content like "About Us" or "Contact."
2. **Articles** → Dynamic content such as news, blogs, or updates.
3. **Custom Content Types** → User-defined structures (e.g., product listings, events).

Q5. [10 Marks each] - Answers

a. Explain MongoDB CRUD Operations with an example.

Definition

MongoDB CRUD stands for **Create, Read, Update, and Delete**. These are the basic operations used to manage data in MongoDB collections and documents.

MongoDB CRUD Operations

1. Create Operation (Insert)

- Used to add new documents into a collection.
- MongoDB provides `insertOne()` and `insertMany()` methods.

Example:

```
db.employees.insertOne({
   name:"Kevin Malone",
   age:57,
   department:"Accountant"
})
```

- Inserts one document into `employees` collection.

---

2. Read Operation (Retrieve)

- Used to fetch documents from a collection.
- Methods used: `find()` and `findOne()`.

Example:

```
db.employees.find({department:"Accountant"})
```

- Displays employees from accountant department.

---

3. Update Operation

- Used to modify existing documents.
- Methods used: `updateOne()`, `updateMany()`.

Example:

```
db.employees.updateOne(
   {name:"Dwight Schrute"},
   {$set:{department:"Sales"}}
)
```

- Updates department field.

---

4. Delete Operation

- Used to remove documents from a collection.
- Methods used: `deleteOne()` and `deleteMany()`.

Example:

```
db.students.deleteOne({name:"Kevin Malone"})
```

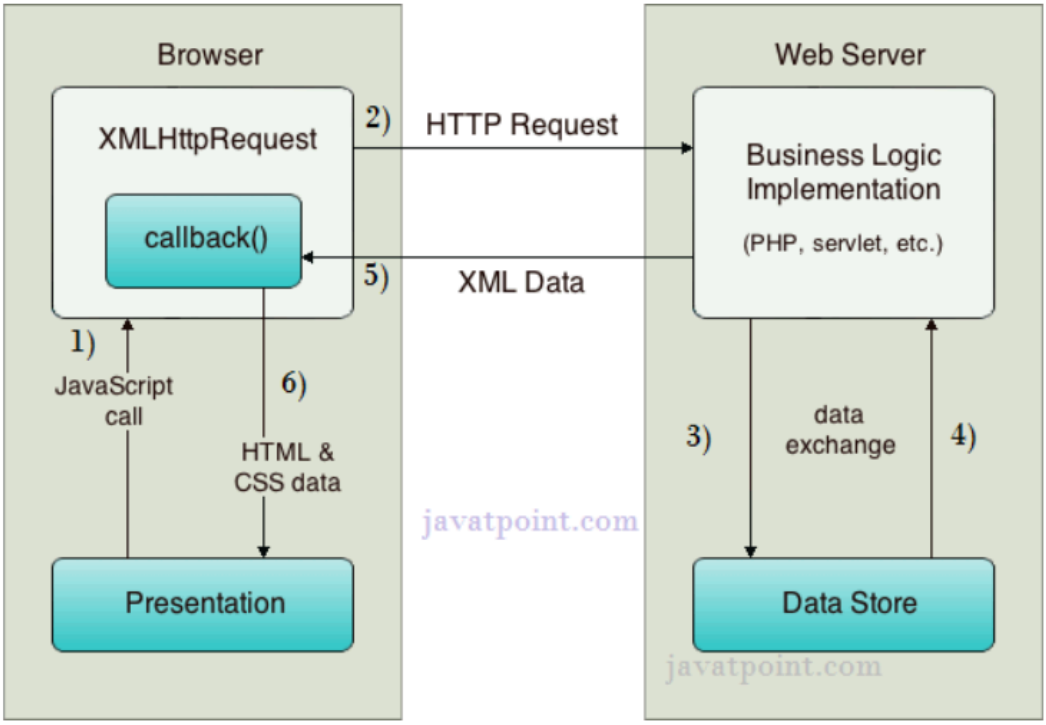
- Deletes the matching document.

Q6. [10 Marks each] - Answers

a. With the help of diagram explain process involved in AJAX Asynchronous Model.

Definition

AJAX (Asynchronous JavaScript and XML) is a technique that allows web pages to communicate with the server **asynchronously**, meaning data can be exchanged in the background without reloading the entire page. This improves user experience by enabling **partial page updates**.



Process of AJAX Working

1. **User Action**
    - A user triggers an event (e.g., button click, form submission).
2. **AJAX Call (JavaScript)**
    - JavaScript creates an **XMLHttpRequest object** and sends a request to the server asynchronously.
3. **Server Processing**
    - The server receives the request, processes it (e.g., fetch data from a database), and prepares a response.
4. **Server Response**
    - The server sends back data in formats like **JSON, XML, or plain text**.
5. **Page Update**
    - JavaScript updates only the relevant part of the page using **DOM manipulation**, without refreshing the entire page.

Example (JavaScript AJAX Call):

```
function loadData() {
  var xhr = new XMLHttpRequest();
  xhr.open("GET", "/data", true); // true → asynchronous
  xhr.onload = function() {
    if (xhr.status == 200) {
      document.getElementById("result").innerHTML = xhr.respo
nseText;
    }
  };
  xhr.send();
}
```

WebX-2024-May-PYQ Answers

Q1. [5 Marks each]

a. Compare and contrast Web 1.0, Web 2.0 and Web 3.0.

b. What is Content Management System (CMS)?

c. Explain steps involved in Web Analytics Process.

d. State the advantages and disadvantages of Joomla.

e. State four significant NTriples language.

Q2. [10 Marks each]

a. What does REST stand for? Explain REST API with a basic flow diagram.

b. Explain the following built-in AngularJS Directives with example:

- a) ng-app
- b) ng-init

Q3. [10 Marks each]

a. Discuss any 5 built-in helper functions in AngularJS.

b. Explain Flask templates with an example.

Q4. [10 Marks each]

a. Explain the working of MongoDB and Applications of MongoDB.

b. With the help of diagram explain process involved in AJAX Asynchronous Model.

Q5. [10 Marks each]

a. What are the HTTP Methods provided by Python Flask?

b. Explain Drupal's architecture with its advantages.

Q6. [10 Marks each]

a. Explain different characteristics of RIA in details.

b. Discuss definite and indefinite loops with suitable examples in Typescript.

Q1. [20 Marks] - Answers

c. Explain steps involved in Web Analytics Process.

1. Definition

Web Analytics Process is the systematic method of collecting, analyzing, and reporting website data to improve performance and user experience.

2. Key Points

- **Step 1: Define Goals** → Identify business objectives (e.g., increase sales, improve engagement).
- **Step 2: Collect Data** → Gather information using tools like Google Analytics (traffic, clicks, conversions).
- **Step 3: Process Data** → Organize raw data into meaningful formats (tables, charts).
- **Step 4: Analyze Data** → Interpret patterns, trends, and user behavior.
- **Step 5: Report Findings** → Present insights in clear reports for decision-making.
- **Step 6: Take Action** → Implement changes (optimize content, improve UI, adjust marketing).
- **Step 7: Review & Repeat** → Continuously monitor and refine strategies.

d. State the advantages and disadvantages of Joomla.

Advantages:

- Open-source and free to use.

- Easy content management through admin panel.
- Supports many extensions and templates.
- Suitable for dynamic and e-commerce websites.

Disadvantages:

- More complex than some CMS platforms for beginners.
- Customization may require technical knowledge.
- Some extensions may have compatibility issues.
- Regular updates and maintenance are needed.

e. State four significant NTriples language.

1. Definition:

N-Triples is a plain text serialization language used in RDF (Resource Description Framework) to represent data in the form of subject–predicate–object triples.

2. Four Significant Features of N-Triples Language:

- **Simple Syntax:** Uses a very simple and easy-to-read text format.
- **Triple-Based Representation:** Represents data as subject, predicate, and object statements.
- **Machine Readable:** Easy for software and RDF parsers to process.
- **Supports Data Exchange:** Used for storing and exchanging semantic web data between systems.

Q4. [10 Marks each] - Answers

a. Explain the working of MongoDB and Applications of MongoDB.

Definition

MongoDB is a **NoSQL, document-oriented database** that stores data in flexible **BSON (Binary JSON)** format. It is widely used for modern applications requiring

scalability, high performance, and schema-less data storage.

Working of MongoDB

1. **Data Storage**
    - Stores data as **documents** in collections (similar to rows in tables).
    - Each document is a JSON-like structure with key-value pairs.
2. **Schema-less Design**
    - No fixed schema; documents in the same collection can have different fields.
3. **CRUD Operations**
    - Supports **Create, Read, Update, Delete** operations using queries.
4. **Indexing**
    - Provides indexes for faster query execution.
5. **Replication**
    - Uses **replica sets** to ensure high availability and fault tolerance.
6. **Sharding**
    - Distributes data across multiple servers for horizontal scalability.
7. **Aggregation Framework**
    - Performs advanced data processing (grouping, filtering, transformations).

Applications of MongoDB

1. **Big Data Applications**
    - Handles large volumes of unstructured/semi-structured data.
2. **Real-Time Analytics**
    - Used in dashboards and monitoring systems for instant insights.
3. **Content Management Systems (CMS)**
    - Flexible schema supports blogs, news portals, and product catalogs.

4. **E-commerce Platforms**
    - Manages product details, customer data, and transactions efficiently.
5. **Social Media Applications**
    - Stores user profiles, posts, comments, and likes dynamically.

Q5. [10 Marks each] - Answers

a. What are the HTTP Methods provided by Python Flask?

Definition

Flask is a lightweight Python web framework that supports **HTTP methods** to handle client-server communication. These methods map to **CRUD operations** and define how data is requested or modified.

HTTP Methods Provided by Flask

1. **GET**
    - Retrieves data from the server.
    - Parameters passed in the URL query string.
    - Example:

```
@app.route('/get', methods=['GET'])
def get_example():
    return "This is a GET request"
```

2. **POST**
    - Sends data to the server (e.g., form submission).
    - Parameters passed in the request body.
    - Example:

```
@app.route('/post', methods=['POST'])
def post_example():
```

```
    return "This is a POST request"
```

3. **PUT**
   - Updates or replaces an existing resource.
   - Example:

```
@app.route('/put', methods=['PUT'])
def put_example():
    return "This is a PUT request"
```

4. **PATCH**
   - Partially updates an existing resource.
   - Example:

```
@app.route('/patch', methods=['PATCH'])
def patch_example():
    return "This is a PATCH request"
```

5. **DELETE**
   - Removes a resource from the server.
   - Example:

```
@app.route('/delete', methods=['DELETE'])
def delete_example():
    return "This is a DELETE request"
```

Q6. [10 Marks each] - Answers

b. Discuss definite and indefinite loops with suitable examples in Typescript.

Definition

In programming, **loops** are used to execute a block of code repeatedly.

- **Definite Loop** → Number of iterations is known in advance.
- **Indefinite Loop** → Number of iterations is not known beforehand; continues until a condition is met.

1. Definite Loop

- Executes a fixed number of times.
- Commonly implemented using a **for loop**.

Example (Definite Loop in TypeScript)

```
for (let i = 1; i <= 5; i++) {
    console.log("Iteration: " + i);
}
```

Explanation:

- Loop runs exactly 5 times.
- Output: Iteration 1 ... Iteration 5.

2. Indefinite Loop

- Executes until a condition becomes false.
- Implemented using **while** or **do-while** loops.

Example (Indefinite Loop in TypeScript)

```
let count = 1;
while (count <= 5) {
    console.log("Count: " + count);
    count++;
}
```

Explanation:

- Loop continues until `count` exceeds 5.
- Number of iterations depends on condition, not fixed beforehand.

WebX-2023-May-PYQ Answers

Q1. [5 Marks each]

a. Explain Semantic Web Stack.

b. Explain an arrow function in TypeScript.

c. What is Routing in AngularJS?

d. Discuss default database in MongoDB: local, admin, and config.

e. Discuss technologies AJAX works with to create interactive web pages.

Q2. [10 Marks each]

a. Define Clickstream Analysis and state its applications.

b. Explain Modules in TypeScript with example.

Q3. [10 Marks each]

a. Explain AngularJS `ng-app`, `ng-init`, `ng-model` directive with examples.

b. Explain Accessing and Manipulating Databases commands in MongoDB.

Q4. [10 Marks each]

a. Explain with example concept of Flask Templates.

b. With diagram explain working of AJAX.

Q5. [10 Marks each]

a. What are the expressions in AngularJS?

b. Explain the concept of Collections and Documents in MongoDB with examples.

Q6. [10 Marks each]

a. What is Flask URL Building?

b. Short note on AngularJS Data Binding.

Q1. [5 Marks each] - Answers

a. Explain Semantic Web Stack.

1. Definition / Introduction:

Semantic Web Stack is a layered architecture of technologies that supports the Semantic Web by enabling data sharing, machine understanding, and intelligent processing.

2. Key Layers of Semantic Web Stack:

- **Unicode and URI:** Provides universal character encoding and unique identification of web resources.
- **XML and XML Schema:** Used for structuring and describing data.
- **RDF (Resource Description Framework):** Represents data in subject–predicate–object triples.
- **RDF Schema (RDFS):** Defines vocabulary and relationships between resources.
- **OWL (Web Ontology Language):** Used to create ontologies and define complex relationships.
- **Logic and Proof Layer:** Supports reasoning and validation of knowledge.
- **Trust Layer:** Ensures reliability, security, and trust in web data.

b. Explain an arrow function in TypeScript.

1. Definition:

Arrow function in TypeScript is a shorter way to write functions using the `=>` symbol. It provides simple syntax and automatically binds `this`.

2. Key Points:

- Uses `=>` (arrow operator) to define functions.
- Provides shorter and cleaner syntax than normal functions.

- Useful for callbacks and anonymous functions.
- Can accept parameters and return values.
- Supports single-line and multi-line function bodies.
- Improves readability and reduces code length.

3. Example:

```
// Arrow function with type annotations
let add = (x: number, y: number): number => x + y;

console.log(add(5, 3)); // Output: 8
```

c. What is Routing in AngularJS?

1. Definition

Routing in AngularJS is the mechanism that allows navigation between different views of an application without reloading the entire page. It enables the creation of Single Page Applications (SPAs).

2. Key Points

- **Single Page Application**: Routing helps load different views dynamically within one page.
- **ngRoute Module**: Provides routing functionality in AngularJS.
- **$routeProvider**: Configures routes by mapping URLs to templates and controllers.
- **Templates & Controllers**: Each route can load a specific HTML template and controller.
- **Seamless Navigation**: Improves user experience by avoiding full page reloads.
- **Dynamic Content**: Different parts of the app can be displayed based on the route.

3. Example

```
// Configure routes
app.config(function($routeProvider) {
    $routeProvider
    .when("/home", {
        templateUrl: "home.html",
        controller: "homeCtrl"
    })
    .when("/about", {
        templateUrl: "about.html",
        controller: "aboutCtrl"
    })
    .otherwise({ redirectTo: "/home" });
});
```

d. Discuss default database in MongoDB: local, admin, and config.

1. Definition:

MongoDB provides some default system databases used for administration, configuration, and internal operations. The main default databases are **local**, **admin**, and **config**.

2. Explanation:

- **admin Database:**
  - It is the primary administrative database in MongoDB.
  - Stores user authentication and administrative information.
  - Used for managing users, roles, and database commands.
- **local Database:**
  - Stores data specific to a single server.
  - Used mainly for replication-related information.
  - This database is not replicated to other servers.

- **config Database:**
  - Stores metadata related to sharding.
  - Keeps configuration settings for sharded clusters.
  - Contains information about shards and data distribution.

3. Example:

- **admin** → User and role management
- **local** → Replication data
- **config** → Sharding configuration data

e. Discuss technologies AJAX works with to create interactive web pages.

1. Definition:

AJAX (Asynchronous JavaScript and XML) is a web technology used to create interactive web pages by updating data without reloading the entire page. It works with multiple technologies together.

2. Technologies Used in AJAX:

- **HTML/XHTML:** Used for creating and structuring web page content.
- **CSS:** Used for styling and designing the web page.
- **JavaScript:** Controls client-side logic and handles AJAX operations.
- **DOM (Document Object Model):** Allows dynamic updating of page content.
- **XMLHttpRequest (XHR):** Used to send and receive data asynchronously from the server.
- **XML/JSON:** Used as data formats for exchanging information between client and server.
- **Server-side Technologies:** Languages like PHP, Java, or ASP.NET process requests and send responses.

Q2. [10 Marks each] - Answers

a. Define Clickstream Analysis and state its applications.

Definition

Clickstream Analysis is the process of tracking and analyzing the sequence of clicks (navigation paths) made by users while browsing a website. It records **pages visited, time spent, entry/exit points, and user actions**, providing insights into user behavior and website performance.

Applications of Clickstream Analysis

1. **User Behavior Analysis**
   - Understand how users navigate through a website.
2. **Website Optimization**
   - Identify popular pages and optimize underperforming ones.
3. **Personalization**
   - Recommend products/content based on browsing history.
4. **Marketing Insights**
   - Track effectiveness of campaigns and advertisements.
5. **Conversion Rate Improvement**
   - Analyze drop-off points in purchase funnels.
6. **Fraud Detection**
   - Detect unusual navigation patterns that may indicate malicious activity.

Example

- A user visits an e-commerce site:
  `Home → Electronics → Mobile Phones → Product Page → Add to Cart → Checkout`
- Clickstream analysis helps identify if users **abandon carts** at checkout, guiding improvements in payment flow.

b. Explain Modules in TypeScript with example.

Definition

In TypeScript, **modules** are used to organize code into reusable and maintainable units. A file containing `import` or `export` statements is treated as a **module**. Modules help avoid **global namespace pollution** and allow developers to share functionality across files.

Features of Modules

- **Encapsulation** → Code is scoped within modules.
- **Reusability** → Functions, classes, and variables can be exported and reused.
- **Maintainability** → Large projects can be split into smaller files.
- **Interoperability** → Works with JavaScript module systems (ES Modules, CommonJS).

Example – External Module

math.ts (Module File)

```
export function add(a: number, b: number): number {
    return a + b;
}

export function multiply(a: number, b: number): number {
    return a * b;
}
```

main.ts (Importing Module)

```
import { add, multiply } from "./math";
```

```
console.log(add(5, 10));        // Output: 15
console.log(multiply(4, 3));    // Output: 12
```

Explanation:

- `math.ts` exports functions `add` and `multiply`.
- `main.ts` imports and uses them, demonstrating modularity.

Q3. [10 Marks each] - Answers

a. Explain AngularJS `ng-app`, `ng-init`, `ng-model` directive with examples.

Definition

AngularJS directives are special HTML attributes that extend the functionality of web pages. Three important built-in directives are `ng-app` (bootstraps the application), `ng-init` (initializes variables), and `ng-model` (binds data between model and view).

a) `ng-app` Directive

- `ng-app` is used to **initialize an AngularJS application**.
- It tells AngularJS where the application starts.
- It is usually placed in the `<html>`, `<body>`, or a `<div>` tag.
- It also loads the AngularJS framework for that section.

Syntax:

```
<div ng-app="">
```

Example:

```
<!DOCTYPE html>
<html ng-app="">
```

```
<body>

<p>{{5 + 5}}</p>

</body>
</html>
```

Output:

10

- Here `ng-app` starts the AngularJS application.
- Angular expression `{{5+5}}` gets evaluated.

b) `ng-init` Directive

- `ng-init` is used to initialize variables or data in AngularJS.
- It assigns initial values before use.
- Commonly used for simple initialization.

Syntax:

```
<div ng-init="name='Angela Martin'">
```

Example

```
<!DOCTYPE html>
<htmlng-app="">
<body>

<divng-init="name='Angela Martin'; marks=85">
   Name: {{name}}<br>
   Marks: {{marks}}
</div>
```

```
</body>
</html>
```

Output:

Name: Angela Martin

Marks: 85

- `ng-init` initializes variables `name` and `marks`.

c) ng-model

- **Purpose**: Creates **two-way data binding** between input fields and variables.
- **Functionality**: Updates the model when the view changes and vice versa.
- **Example**:

```
<div ng-app="">
  <p>Enter Salary: <input type="number" ng-model="salary"></p
>
  <p>Your Salary is: {{salary}}</p>
</div>
```

Explanation:

- `ng-model` binds the input field to the variable salary.
- Any change in the input instantly reflects in the output.

b. Explain Accessing and Manipulating Databases commands in MongoDB.

1. Accessing Databases

- **Show Databases** → Lists all available databases.

```
show dbs
```

- **Use Database** → Switches to a specific database (creates if not exists).

```
use myDatabase
```

- **Show Collections** → Lists all collections in the current database.

```
show collections
```

2. Manipulating Databases (CRUD Commands)

a) Create (Insert Data)

```
db.employees.insertOne({ name: "Stanley Hudson", department:
"Sales", year: 26 });
```

- Adds a new document to the `employees` collection.

b) Read (Retrieve Data)

```
db.employees.find({ name: "Stanley Hudson" });
```

- Retrieves documents matching the condition.

c) Update (Modify Data)

```
db.employees.updateOne(
  { name: "Stanley Hudson" },
  { $set: { year: 27 } }
);
```

- Updates the `year` field of Stanley's record.

d) Delete (Remove Data)

```
db.employees.deleteOne({ name: "Stanley Hudson" });
```

- Deletes Stanley's record from the collection.

Q4. [10 Marks each] - Answers

b. Explain working of AJAX with diagram? How XMLHttpRequest object works in AJAX?

For working of AJAX with diagram answer refer PYQ of 2025 December (Q6→b)

Working of XMLHttpRequest Object in AJAX

- **Creation**:

```
var xhr = new XMLHttpRequest();
```

- **Open Connection**:

```
xhr.open("GET", "/data", true); // true → asynchronous
```

- **Send Request**:

```
xhr.send();
```

- **Handle Response**:

```
xhr.onload = function() {
  if (xhr.status == 200) {
    document.getElementById("result").innerHTML = xhr.resp
onseText;
  }
};
```

Explanation:

- `open()` → Initializes request (method, URL, async flag).

- `send()` → Sends request to server.
- `onload` → Handles server response when ready.
- `responseText` → Contains returned data.

Q5. [10 Marks] - Answers

a. What are the expressions in AngularJS?

Definition

In AngularJS, **expressions** are snippets of code written inside double curly braces `{{ }}`. They are used to **bind data** between the model and the view. Expressions evaluate values from the scope and display them dynamically in the HTML page.

Features of AngularJS Expressions

- Written inside `{{ }}` brackets.
- Can contain variables, operators, and function calls.
- Automatically update when the scope data changes (**two-way binding**).
- Cannot contain control flow statements (like `if`, `for`).
- Similar to JavaScript expressions but evaluated by AngularJS scope.

Examples

1. Displaying Variables

```
<div ng-app="" ng-init="name='Michael Scott'; age=60">
  <p>Name: {{name}}</p>
  <p>Age: {{age}}</p>
</div>
```

2. Arithmetic Expression

```
<div ng-app="" ng-init="x=10; y=20">
  <p>Sum: {{x + y}}</p>
</div>
```

3. Function Call Expression

```
<div ng-app="" ng-init="message='Hello AngularJS'">
  <p>Length: {{message.length}}</p>
</div>
```

b. Explain the concept of Collections and Documents in MongoDB with examples.

Definition

MongoDB is a **NoSQL document-oriented database**. Its data is organized into **collections** (similar to tables in RDBMS) and **documents** (similar to rows, but more flexible).

1. Collections

- A **collection** is a group of MongoDB documents.
- Equivalent to a table in relational databases.
- Collections do not enforce a schema, meaning documents can have different structures.

Example – Creating a Collection

```
use myDatabase
db.createCollection("employees")
```

This creates a collection named **employees** inside `myDatabase`.

2. Documents

- A **document** is a record in MongoDB stored in **BSON (Binary JSON)** format.

- Each document contains key-value pairs.
- Documents inside the same collection can have different fields.

Example – Inserting a Document

```
db.employees.insertOne({
    name: "Oscar Martinez",
    department: "Accountant",
    year: 26,
})
```

This inserts a document into the employee's collection.

Q6. [10 Marks each] - Answers

a. What is Flask URL Building?

Definition

In Flask, **URL Building** refers to the process of **generating URLs dynamically** for specific functions instead of hardcoding them. This ensures that links remain valid even if route definitions change, making applications more flexible and maintainable.

Purpose of URL Building

- Avoids hardcoding URLs.
- Ensures consistency when routes are modified.
- Simplifies linking between different parts of the application.
- Uses the `url_for()` function to generate URLs.

Example – Flask URL Building

```
from flask import Flask, url_for
```

```python
app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to Home Page"

@app.route('/about')
def about():
    return "About Page"

@app.route('/links')
def links():
    # Dynamically generate URLs
    home_url = url_for('home')
    about_url = url_for('about')
    return f"Home URL: {home_url} <br> About URL: {about_ur
l}"

if __name__ == '__main__':
    app.run(debug=True)
```

Explanation:

- `url_for('home')` → Generates URL for the `home` function.
- `url_for('about')` → Generates URL for the `about` function.
- Even if route paths change, `url_for()` ensures links remain valid.

b. short note on AngularJS Data Binding.

Definition

Data Binding in AngularJS is the process of creating a **connection between the model (JavaScript objects) and the view (HTML elements)**. It ensures that changes in the model are automatically reflected in the view, and vice versa, depending on the type of binding.

Types of Data Binding in AngularJS

1. **One-Way Data Binding**
   - Data flows **from model to view** only.
   - Example:

```
<div ng-app="" ng-init="course='Web X.0'">
  <p>Course: {{course}}</p>
</div>
```

   Explanation: The value of `course` is displayed in the view, but changes in the view do not affect the model.

2. **Two-Way Data Binding**
   - Data flows **both ways**: from model to view and from view to model.
   - Achieved using the `ng-model` **directive**.
   - Example:

```
<div ng-app="">
  <p>Enter Name: <input type="text" ng-model="student"></p
>
  <p>Hello {{student}}</p>
</div>
```

   Explanation: If the user types in the input field, the model (`student`) updates, and the change is instantly reflected in the view.

Advantages of Data Binding

- Reduces manual DOM manipulation.
- Makes applications **interactive and responsive**.
- Simplifies synchronization between UI and data.

WebX-2023-December-PYQ Answers

Q1. [5 Marks each]

a. Explain Multilevel Inheritance in Typescript

b. Illustrate AngularJS custom directives with suitable example

c. Explain Characteristics of Semantic Web

d. What are the features of Python Flask?

e. Differentiate between CMS and Framework

Q2. [10 Marks each]

a. Explain controllers in Angular JS with example

b. Explain MongoDB Data Types along with syntax

Q3. [10 Marks each]

a. Compare and contrast Web 1.0, Web 2.0 and Web 3.0
b. Explain Routing using ng-Route, ng-Repeat, ng-style, ng-view. With suitable example

Q4. [10 Marks each]

a. Define RIA and explain characteristics of RIA

b. Differentiate Var v/s Let. In addition, Explain function overloading with suitable examples

Q5. [10 Marks each]

a. Explain MongoDB CRUD Operations with an example

b. How to set access and delete cookies in Python Flask

Q6. [10 Marks each]

a. Explain features and Working of AJAX

b. Explain AngularJS $http service in detail with its get() and post() methods

Q1. [5 Marks each] - Answers

a. Explain Multilevel Inheritance in Typescript

1. Definition:

Multilevel Inheritance in TypeScript is a type of inheritance where one class inherits from another class, and a third class inherits from the second class, forming a chain of inheritance.

2. Key Points:

- It involves more than one level of parent-child relationship.
- A child class can inherit properties and methods of its parent and grandparent classes.
- It promotes code reusability.
- It reduces duplication by sharing common features.
- TypeScript uses the `extends` keyword for inheritance.
- Derived classes can add their own properties and methods.

3. Example:

```
class Animal {
   eat() {
      console.log("Eating");
   }
}

class Dog extends Animal {
   bark() {
```

```
        console.log("Barking");
    }
}

class Puppy extends Dog {
    weep() {
        console.log("Weeping");
    }
}
```

- `Dog` inherits from `Animal`
- `Puppy` inherits from `Dog`
- `Puppy` can access methods of both parent classes.

b. Illustrate AngularJS custom directives with suitable example

1. Definition:

AngularJS Custom Directives are user-defined directives used to create reusable HTML components or add custom behavior to elements in an AngularJS application.

2. Key Points:

- Custom directives are created using the `directive()` method.
- They allow developers to define their own HTML tags or attributes.
- Used to extend HTML functionality.
- Help in code reusability and modular design.
- Can manipulate DOM elements and behavior.
- Directives can be used as elements, attributes, classes, or comments.
- Commonly used for reusable UI components.

3. Example:

```
// Define a custom directive
var app = angular.module("myApp", []);

app.directive("studentInfo", function() {
    return {
        template: "<h3>Student Name: {{name}}</h3><p>Course:
{{course}}</p>"
    };
});

app.controller("studentCtrl", function($scope) {
    $scope.name = "Pam Beesley";
    $scope.course = "Art";
});
```

c. Explain Characteristics of Semantic Web

1. Definition / Introduction:

Semantic Web is an extension of the World Wide Web in which data is structured and meaningful so that both humans and machines can understand and process it.

2. Characteristics of Semantic Web:

- **Machine Understandable Data:** Data is organized so machines can interpret meaning.
- **Metadata Based:** Uses metadata to describe information and relationships.
- **Interoperability:** Enables data sharing between different systems and applications.
- **Use of Ontologies:** Defines concepts and relationships using ontologies.
- **Intelligent Search:** Improves search results by understanding context and meaning.
- **Linked Data:** Connects related data from multiple sources on the web.

- **Automation and Reasoning:** Supports automated decision-making and logical reasoning.

d. What are the features of Python Flask?

1. Definition:

Flask is a lightweight Python web framework used for developing web applications. It is simple, flexible, and easy to use.

2. Features of Python Flask:

- **Lightweight Framework:** Minimal and simple framework with less overhead.
- **Built-in Development Server:** Provides an inbuilt server for testing applications.
- **Routing Support:** Supports URL routing for handling web requests.
- **Template Engine:** Uses Jinja2 template engine for dynamic web pages.
- **RESTful Request Handling:** Suitable for creating APIs and web services.
- **Flexible and Scalable:** Easy to customize and can grow with application needs.

e. Differentiate between CMS and Framework

Feature	CMS	Framework
Meaning	Content Management System	Software development structure
Purpose	Manage website content	Develop web applications
Coding Requirement	Little or no coding needed	Requires programming knowledge
Flexibility	Limited customization	Highly customizable
Development Speed	Faster for ready-made sites	Takes more development time
User Type	Suitable for non-programmers	Mainly used by developers
Examples	WordPress, Joomla	Django, Laravel

Q3. [10 Marks each] - Answers

b. Explain Routing using ng-Route, ng-Repeat, ng-style, ng-view. With suitable example

Introduction

AngularJS provides powerful directives to build **Single Page Applications (SPAs)**, handle **dynamic lists**, apply **styles dynamically**, and manage **views**. Let's break them down with examples.

1. ng-route

- **Purpose**: Enables routing in AngularJS applications.
- **Functionality**: Maps URLs to templates and controllers using `$routeProvider`.
- **Example**:

```
<div ng-app="myApp">
  <div ng-view></div>
</div>

<script>
var app = angular.module("myApp", ["ngRoute"]);
app.config(function($routeProvider) {
  $routeProvider
    .when("/home", { templateUrl: "home.html" })
    .when("/about", { templateUrl: "about.html" })
    .otherwise({ redirectTo: "/home" });
});
</script>
```

Explanation: Navigating to `/home` or `/about` loads respective templates into `ng-view`.

2. ng-repeat

- **Purpose**: Iterates over a collection (array/object) and displays items.

- **Example**:

```
<div ng-app="" ng-init="employees=['Jim','Pam','Dwight']">
  <ul>
    <li ng-repeat="s in employees">{{s}}</li>
  </ul>
</div>
```

3. ng-style

- **Purpose**: Dynamically applies CSS styles based on expressions.
- **Example**:

```
<div ng-app="" ng-init="myColor='blue'; mySize='20px'">
  <p ng-style="{'color': myColor, 'font-size': mySize}">
    This text is styled dynamically!
  </p>
</div>
```

Explanation: The text color and size are controlled by AngularJS variables.

4. ng-view

- **Purpose**: Acts as a placeholder where routed templates are displayed.
- **Usage**: Always used with `ng-route`.
- **Example**:

```
<body ng-app="myApp">
  <a href="#!/home">Home</a>
  <a href="#!/about">About</a>
  <div ng-view></div>
</body>
```

Explanation: When a route is triggered, the corresponding template is injected into `ng-view`.

Q4. [10 Marks each] - Answers

b. Differentiate Var v/s Let. In addition, Explain function overloading with suitable examples

Basis	var	let
Scope	Function scoped	Block scoped
Redeclaration	Allowed	Not allowed in same scope
Reassignment	Allowed	Allowed
Hoisting	Hoisted and initialized with undefined	Hoisted but not initialized
Use in Loops	Can cause scope issues	Safer in loops
Introduced In	JavaScript older versions	ES6 / TypeScript

Example Demonstration:

```
function demo() {
    if (true) {
        var a = 10;
        let b = 20;
    }
    console.log(a); // Works (function scope)
    console.log(b); // Error (block scope)
}
```

Part B: Function Overloading in TypeScript

Definition

Function Overloading allows multiple function signatures with the same name but different parameter types or counts. It improves flexibility and readability.

Example – Function Overloading

```
// Function Overload Signatures
function greet(name: string): string;
function greet(age: number): string;

// Function Implementation
function greet(value: any): string {
    if (typeof value === "string") {
        return "Hello, " + value;
    } else if (typeof value === "number") {
        return "You are " + value + " years old.";
    }
    return "Invalid input";
}

// Usage
console.log(greet("Pam"));   // Output: Hello, Sameer
console.log(greet(25));         // Output: You are 25 years o
ld.
```

Q6. [10 Marks each] - Answers

a. Explain AngularJS $http service in detail with its get() and post() methods

Definition

The **$http service** in AngularJS is used to make **asynchronous HTTP requests** (AJAX calls) to remote servers. It allows communication between the client and server, enabling data retrieval or submission without reloading the entire page.

Features of $http Service

- Provides methods like **GET, POST, PUT, DELETE**.
- Returns a **promise object** (`then` , `success` , `error`).

- Supports **JSON data exchange**.
- Useful for building **RESTful APIs** in AngularJS applications.

1. GET Method

- **Purpose**: Retrieve data from the server.
- **Syntax**:

```
$http.get("server-url")
     .then(function(response) {
         $scope.data = response.data;
     });
```

Example:

```
app.controller("myCtrl", function($scope, $http) {
    $http.get("students.json")
         .then(function(response) {
             $scope.students = response.data;
         });
});
```

Explanation: Fetches data from `students.json` and stores it in `$scope.students`.

2. POST Method

- **Purpose**: Send data to the server (e.g., form submission).
- **Syntax**:

```
$http.post("server-url", dataObject)
     .then(function(response) {
         $scope.result = response.data;
     });
```

Example:

```
app.controller("formCtrl", function($scope, $http) {
    $scope.submitForm = function() {
        var student = { name: $scope.name, course: $scope.cou
rse };
        $http.post("/addStudent", student)
             .then(function(response) {
                 $scope.message = response.data;
             });
    };
});
```

Explanation: Sends student data to `/addStudent` endpoint and displays server response.

Workflow of $http Service

1. Client triggers request using `$http`.
2. Request sent to server (GET/POST).
3. Server processes and responds with data.
4. AngularJS updates the view using `$scope`.